AI SYSTEMS SCIENCE / AGENT SYSTEMS

32讲 · 零基础理论课程 • 第二部分：基础技术

06

大语言模型怎样利用上下文信息？

How Language Models Use Context: Attention, Q / K / V and Agent Memory

从Token和向量开始，真正算清注意力权重、因果掩码与多层Transformer；理解模型内部计算为什么不等于智能体长期记忆。

零基础 · 逐步推导 | 只讲理论 | 13张原理图 · 12道思考题

COURSE ROADMAP

本讲学习地图

把“向量如何互相关联”一步步算出来，再将模型内部机制放回完整Agent架构。

00	本讲学习目标：让“上下文在模型里动起来”
01	1. 从科研助手的错误回答开始：为什么看到的信息没有被正确使用?
02	2. 第一步：文字为什么能进入数学计算?
03	3. 第二步：为什么要让一个Token参考另一个Token?
04	4. Q、K、V到底是什么？从一次计算理解它们
05	5. 从相似度得分到注意力权重：真正算一次
06	6. 为什么生成式语言模型需要因果掩码?
07	7. 一种注意力关系不够吗？多头注意力在做什么?
08	8. 位置如何影响注意力？为什么同样的事实换个地方会有区别?
09	9. 预填充、逐Token生成和KV Cache到底发生了什么?
10	10. 注意力是一种“计算中的信息混合”，不是外部记忆检索
11	11. 让一个科研案例“从头到尾动一次”
12	12. 如果出现错误，应从哪一层开始排查?
13	13. 常见误区：逐条对照消除混淆
14	14. 理论思考题：先自己推导，不要急着看答案
15	15. 思考题参考答案：重点是推导逻辑
16	16. 把第05、06讲连接成一张完整的知识地图
17	17. 学习建议：不背词，尝试在脑中重放一次计算
18	18. 核对过的原始研究文献与延伸阅读

贯穿主线

外部检索 → 组装上下文 → 向量化 → Q/K/V注意力 → 输出 → 反馈

第06讲 大语言模型怎样利用上下文信息?

课程：《智能体记忆系统：从零基础到科研理解》（32讲理论课程）
单元：第二部分 · 理解记忆系统必需的基础技术（第2讲）
讲义版本：v1.0 · 2026年10月18日
定位：零基础、只讲理论；不需要微积分、线性代数基础或编程经验。公式只用于把操作过程看清楚。
承接：第05讲知道“一个窗口能装下多少Token”；第06讲进一步回答：**装进去以后，模型究竟怎样在这些Token之间建立联系？为什么装进去了，还可能用错?**

本讲学习目标：让“上下文在模型里动起来”

这一讲不要求你记住复杂公式，而是要能独立推演六件事：①文本怎样变成可计算的向量；②为什么一个Token需要参考前面的Token；③Q、K、V分别扮演什么计算角色；④相似度得分怎样变成注意力权重；⑤为什么生成式模型不能在当前位置偷看未来Token；⑥为什么**模型内部的注意力**与**智能体外部的记忆检索**是两种不同的机制。

> **先建立整图**：应用把当前问题、检索回来的历史事实、最近对话与必要指令组成输入 → Tokenizer变成Token ID → 模型把它们变为向量并加入位置关系 → 多层Transformer通过注意力交换信息 → 产生下一个Token的概率分布 → 逐步生成答案；若需要长期记忆，仍要靠模型外部的控制器进行存储、查找和更新。**“看见信息”不是“正确使用信息”的保证。**

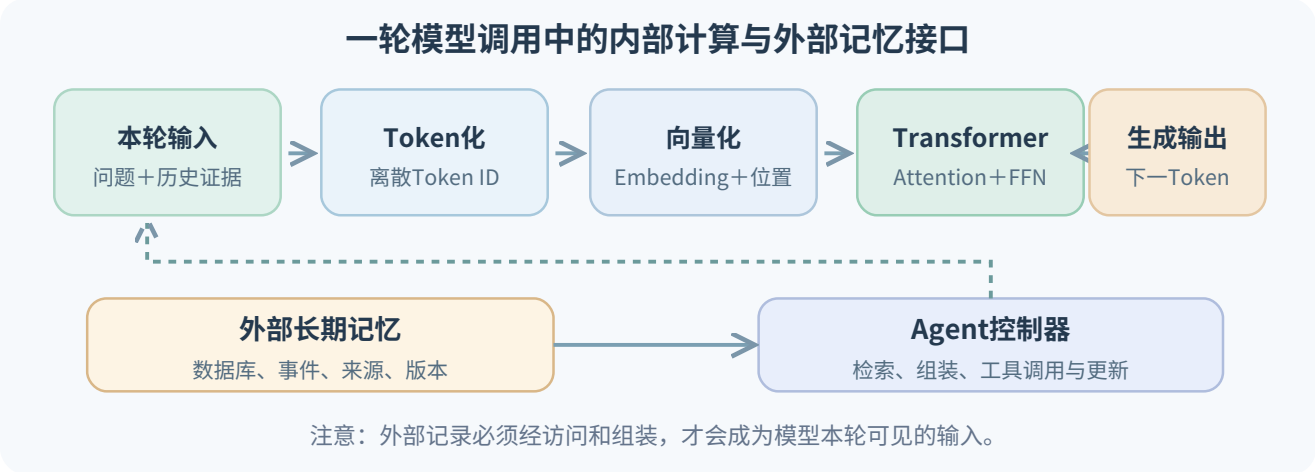


图1 · 跨时间记忆与模型内部注意力各有不同的职责。

1. 从科研助手的错误回答开始：为什么看到的信息没有被正确使用？

假设一个虚构的科研助手，正在协助研究一个长期项目。前期曾决定先研究**记忆数量越多是否越好**；后来查阅资料后发现，过时记忆可能干扰决策，于是在第18天把这一假设改为“待重新评估”，第21天明确决定“暂不继续简单增加记忆数量”。

第30天，系统把旧假设、反对证据和新决定**都放进了本轮上下文**。研究者问：“下一步首先检查什么？”然而，模型仍然回答：“应该继续扩充记忆数量。”

请注意：这次不能简单归因于“系统忘了”。因为所有必要文字都进入了输入。至少有几种不同的可能：模型没有有效关联“暂不继续”与旧方案；模型错误理解了时间顺序；其他文本让错误方案看起来更加显著；或模型虽识别事实，却在形成建议时进行了不合理推断。

这就引出一个底层问题：**模型的上下文不是一份人类按页阅读的文档。它会把Token转换成数字表示，反复计算这些位置之间的关系。** 这种关系计算中最重要的模块之一，就是注意力机制（Attention）。

问题	发生在哪一层	可能的表现
旧决定根本未被放进输入	外部检索 / 上下文组装	模型没有任何机会使用新决定
旧、新决定都在，但没有正确联系	模型内部表征与推理	忽略时间、否定和条件信息
把历史意见当成当前命令	信息权限与来源解释	旧草稿覆盖最近的正式决定
正确识别事实，却给出错误建议	推理 / 决策	“理解了事实”不等于“行动合理”

学习提醒：我们能通过输入与输出来观察错误，但一般不能只凭一次回答就精确判定模型内部某个注意力头“没有关注到”哪条信息。后面会专门解释这个证据边界。

2. 第一步：文字为什么能进入数学计算？

上一讲介绍过Tokenizer：它把文字切成Token并编码为离散的ID，例如把一个句子变成 $t_1, t_2, t_3, \ldots, t_n$。这里的 t 只代表第几个Token，并不等于中文字数，也不说明词语重要性。

2.1 Token ID不是词义，需要变成向量

模型不会拿“第100号Token”和“第120号Token”的编号直接比较。它需要先查一张通过训练学到的参数表，把每个Token映射成一组数字。例如下面只是二维示意，不代表任何真实模型的数据：

- `旧计划` → `[0.2, 0.8]`
- `被否决` → `[0.9, 0.1]`
- `新决定` → `[0.6, 0.7]`

这样的一组数字叫**向量（Vector）**；把Token映射成向量表示的步骤叫**嵌入（Embedding）**。真实模型的向量维度通常高得多，二维数字仅用来让你看见计算的形状；不能把两个坐标分别解释为“重要程度”和“真实性”。向量的具体意义由训练以及后续模型计算共同决定。

2.2 同一个词为何在不同句子里含义不同?

例如“苹果”可能指水果，也可能指公司。初始的Token嵌入可以相同或相近，但当它与“水果店”或“手机发布会”等其他Token一起进入模型后，各层注意力及前馈网络会改变它在上下文中的隐藏表示。最后得到的是受周围信息影响的**上下文化表示（Contextualized representation）**。

这里还要区别**初始Embedding**与**Transformer中间层的Hidden State（隐藏状态）**：前者是对输入符号的初步表示，后者是经过多层计算、不断融合上下文的结果。Q、K、V通常就是从某一层的隐藏状态通过不同矩阵投影获得的，并非直接从中文句子手工查三张表。

2.3 光有内容向量，为什么还不够?

“甲否决乙”和“乙否决甲”包含近似相同的词，却完全不是同一事实。模型必须在一定程度上处理**顺序和位置**，否则光看有哪些词无法重建句意。Transformer会通过某种位置表示机制让位置关系参与计算；具体实现因模型而异，比如绝对位置编码、相对位置设计、旋转位置嵌入（RoPE）等。

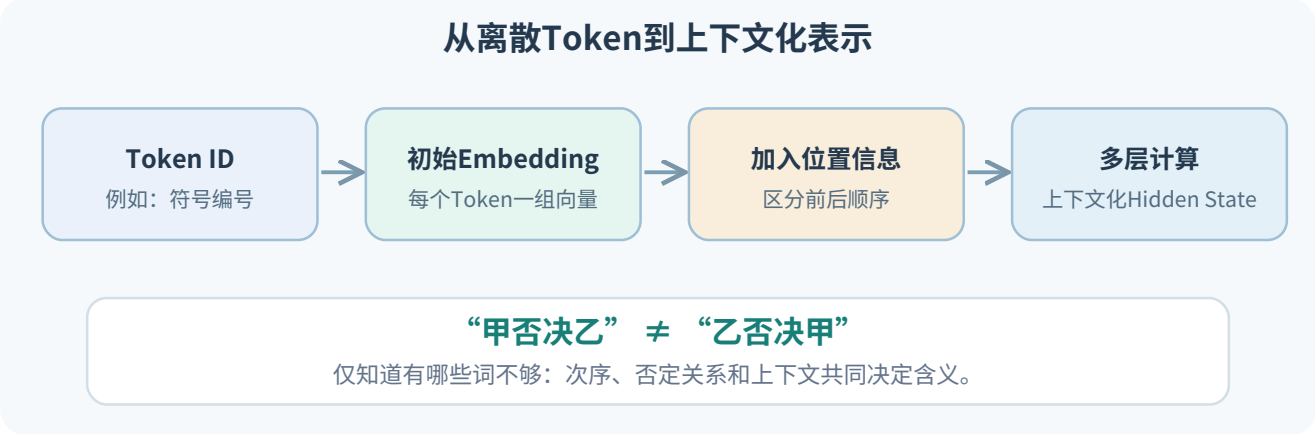


图2 · Embedding是起点，多层隐藏状态才能逐步结合上下文。

推导结论：Token告诉系统“有哪些离散符号”；Embedding提供可计算的初始向量；位置信息帮助表示先后关系；注意力和后续网络层进一步处理“这些符号在这个上下文里彼此意味着什么”。

3. 第二步：为什么要让一个Token参考另一个Token?

先比较两句话：

- “研究者**没有批准**扩大实验。”
- “研究者**已经批准**扩大实验。”

如果模型在处理“扩大实验”时只看这几个字，就很难分辨当前状态。但若同时利用前面的“没有批准”或“已经批准”，含义就不同。处理代词、否定词、指代关系、时间修订和跨段落因果时，这个问题尤其突出。

3.1 注意力机制的直观任务

对当前位置的某个Token，注意力模块会计算：在当前允许访问的其他位置中，哪些位置的表示应当以多大权重参与更新当前表示？

注意力不是一个会说话的小人，在人类意义上决定“我想看第几页”。它是一组可训练的矩阵运算。**权重分配是连续数值，不等于模型认定某条事实真实或权威。**

3.2 用“资料卡”理解，但不要把比喻当成实现

假设你需要找“今天的最新研究决定”，桌上有三张资料卡：A是第7天的旧方案，B是第18天的反对证据，C是第21天的最终决定。你可能先形成一个“我要找什么”的问题，再观察每张卡有没有匹配的标题，然后读取匹配资料的内容。

这个比喻与Q/K/V相似，但有重要不同：**真正的Q、K、V都是高维数字向量，由模型内部的参数矩阵计算出来；它们不是硬盘文件、数据库字段，也不是程序员写的自然语言搜索词。**

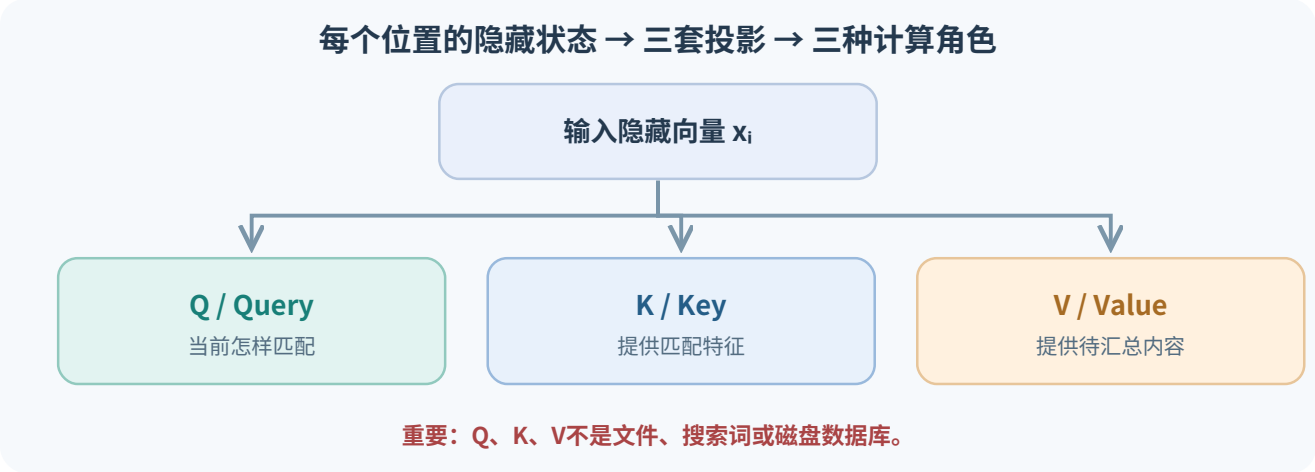


图3 · Q、K与V都属于同一层内部的可学习向量投影。

4. Q、K、V到底是什么？从一次计算理解它们

在一个典型的自注意力层（Self-Attention）中，输入是一组隐藏状态矩阵 `X`。模型拥有三组通过训练得到的投影矩阵：`W_Q`、`W_K`、`W_V`。计算过程可以写成：

`Q = X W_Q` `K = X W_K` `V = X W_V`

这里 `W` 只是矩阵（很多可学习的数字），不是独立运行的软件模块。对每个位置来说，q_i、k_i、v_i 分别代表它在本层、这个注意力头中的查询、键、值向量。

符号	英文	可以怎样理解	不应怎样误解
Q	Query，查询向量	当前位置用什么数值模式匹配可访问位置	用户输入的搜索句子本身
K	Key，键向量	每个位置提供可被匹配的数值特征	磁盘上的文件名或向量数据库索引
V	Value，值向量	被加权汇总、用来更新位置表示的数值内容	一条未经处理的中文记忆记录

4.1 为什么Q与K不直接使用X?

让不同的学习矩阵产生Q和K，模型就可以学会“以某种方式提出匹配需求”及“以另一种方式描述候选位置”。比如同样的一个词，在“谁执行动作”和“动作作用到谁”的关系中，可能需要不同的匹配模式。**这不是说某个头一定在学语法、另一个头一定在学事实**：实际分工需要分析证据，不能凭比喻擅自断言。

4.2 自注意力为什么叫“自”？

因为这些Q、K、V通常来自同一序列在本层的表示：本轮输入中的各个位置互相交换信息。这里的“自”指**同一序列内**，不等于“只看自己一个Token”。

与它相对，**交叉注意力（Cross-Attention）**允许查询来自一个序列，键和值来自另一个序列。在原始Transformer的编码器—解码器架构中这很常见；许多现代仅解码器的LLM主要采用带因果掩码的自注意力，但具体模型可以有变体。[1]

5. 从相似度得分到注意力权重：真正算一次

这部分是全讲最重要的数学例子。为了可以亲手复算，我们**虚构**三个历史信息位置A、B、C。真实模型不会使用下面的人为整数，也不能据此推断实际语言意义。

假设当前位置的查询向量为：`q = [1, 1]`。

三个位置的键向量分别为：

- A（旧方案）：`k_A = [1, 0]`
- B（反对证据）：`k_B = [0, 1]`
- C（新决定）：`k_C = [1, 1]`

5.1 第一步：用点积得到匹配分数

点积（Dot Product）就是把对应位置的数字相乘，再相加。例如 `[1,1] · [1,0] = 1×1 + 1×0 = 1`。

于是：

- `q · k_A = 1`
- `q · k_B = 1`

- `q · k_C = 2`

从这个构造的例子可以看到：在这一组人为选定的向量中，位置C的匹配分数最高。注意：**匹配分数高只是模型学习到的数值关系，不足以证明C的事实更可靠。**

5.2 第二步：为什么要除以sqrt(d_k)?

如果键和查询向量维度变大，点积的典型数值尺度可能随维度增加而变大，影响后面的softmax分布与优化。经典**缩放点积注意力（Scaled Dot-Product Attention）**因此采用除以 `sqrt(dₖ)` 的操作；其中 `dₖ` 是键/查询向量的维度。[1]

本例有两个维度，所以 `sqrt(dₖ)=sqrt(2)≈1.414`。得到：

- A：`1/1.414 ≈ 0.707`
- B：`1/1.414 ≈ 0.707`
- C：`2/1.414 ≈ 1.414`

5.3 第三步：softmax把分数变成权重

Softmax的作用是：将一组分数转换为非负的相对权重，这些权重之和为1。公式是：

```
weight_i = exp(score_i) / [exp(score_A) + exp(score_B) + exp(score_C)]
```

`exp()` 是以自然常数 `e≈2.718` 为底的指数函数。你不用记函数图像，知道它会把较高的分数转换成相对更大的正数即可。

用上面三个分数计算后（四舍五入）：

位置	缩放分数	Softmax权重	通俗理解
A，旧方案	0.707	**0.248**	在本例中占约24.8%
B，反对证据	0.707	**0.248**	在本例中占约24.8%
C，新决定	1.414	**0.503**	在本例中占约50.3%

总和约为 `0.999`，偏离1的极小差值来自小数四舍五入。精确计算时总和为1。

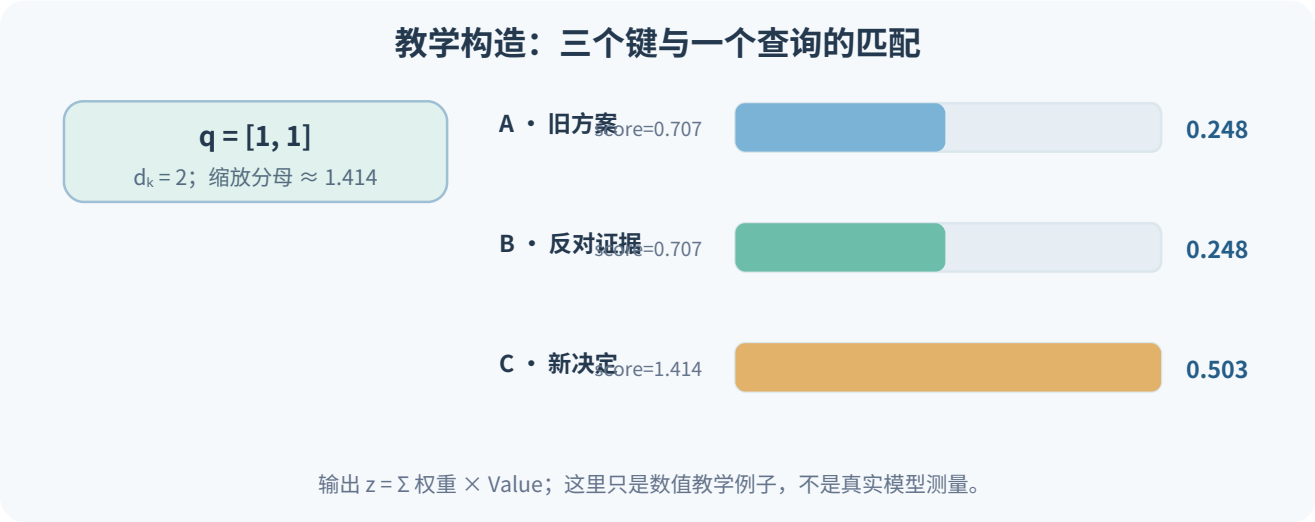


图4 · Softmax把匹配分数转化为相对权重，随后对V加权求和。

5.4 第四步：不是选一个Token，而是对V做加权求和

我们再给三个位置选取**虚构的Value向量**：

`v_A = [1,0]`，`v_B = [0,1]`，`v_C = [1,1]`。

输出向量是：

`z = 0.248 · v_A + 0.248 · v_B + 0.503 · v_C`

因此：

`z ≈ [0.248+0.503, 0.248+0.503] = [0.751, 0.751]`。

这才是注意力的关键：匹配用Q和K，实际汇总用V。输出不是“把C原文复制过来”，也不是“必然把C当作最终决定”，而是形成新的数字表示，供后续网络层继续计算。

5.5 把一整行、一整层写成统一公式

$$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}(QK^{\top}/\sqrt{d_k} + M)\,V$$

其中：

- $QK^{\top}$：计算每个查询位置与键位置的匹配分数；
- $\sqrt{d_k}$：经典缩放因子；
- M：可选的掩码（Mask），用来排除当前不允许访问的位置；
- $\mathrm{softmax}$：归一化成每一行的权重；
- 最后乘 V：对可访问位置的值向量做加权汇总。

后面讲“因果掩码”时，这个 M 将发挥决定性作用。公式形式代表**经典注意力的核心计算**；现实模型可能采用其他注意力变体、不同的归一化和具体实现优化。

6. 为什么生成式语言模型需要因果掩码?

假设模型正在按顺序生成：“新的研究决定是……” 对于第4个位置，模型只能依据它目前已有的输入以及之前已生成的Token，不能把“未来尚未生成的答案”偷看进来。

因果掩码（Causal Mask）的基本规则是：位置 `i` 通常可以访问位置 `j≤i`，而不能访问 `j>i`。这让模型在训练的下一Token预测和推理时符合自回归生成的因果方向。



图5 · 标准自回归注意力的可见性呈下三角形。

6.1 用4个位置推演

设序列为：①“旧”、②“方案”、③“已”、④“取消”。

- 第①位置只能通过该头参考第①位置；
- 第②位置可参考第①、②位置；
- 第③位置可参考第①、②、③位置；
- 第④位置可参考全部四个已出现的位置。

表格左下方允许访问，右上方遮蔽。实现上可给未来位置分数加一个极大的负数（理论表述常写 `−∞`），让softmax后的权重趋近或等于0。

6.2 一个易错理解：用户问题放在最后，前面资料能“看到”问题吗?

在标准因果自注意力中，**较早位置的表示不会因为后面新追加的问题而在同一轮前向计算中反过来访问未来位置**。但问题位置本身出现在后面，因此它及之后生成的Token可以访问前面的资料表示。多层网络可以据此从先前信息中提取与当前问题相关的内容。

这正是问答中常见的输入组织方法：先放资料，再放“请根据上述资料回答……”；模型的生成位置在后面，可以参考前面的资料。这不等于所有模型都是这样训练，也不说明这种排列在任何任务下都最好。

6.3 注意“生成一个Token”和“处理已有输入”

对于一段已经提供的提示词，模型一般会先对整段输入进行**预填充（Prefill）**计算；之后依次生成Token，每生成一个新的Token，序列继续向右扩展。在带因果掩码的架构里，前面的表示和KV可被缓存复用，而新位置可以对之前位置计算注意力。下一节会解释为什么这并不是外部长期记忆。

7. 一种注意力关系不够吗？多头注意力在做什么？

多头注意力（Multi-Head Attention）把隐藏表示投影到多组不同的Q/K/V子空间，各头分别计算注意力，再把多头结果拼接并经过输出投影。不同的头可以提供不同的表示与交互方式。[1]

举个教学上容易理解的例子：为了处理“第7天提出方案，第21天因新证据取消方案”，系统需要结合否定关系、时间关系、实体指代和任务目标。多个注意力头提供了表达不同关系的可能性，**但并不意味着每个头都固定专门负责其中一个人类定义的功能**。不能根据某个头的可视化直接宣布它“就是时间理解头”。

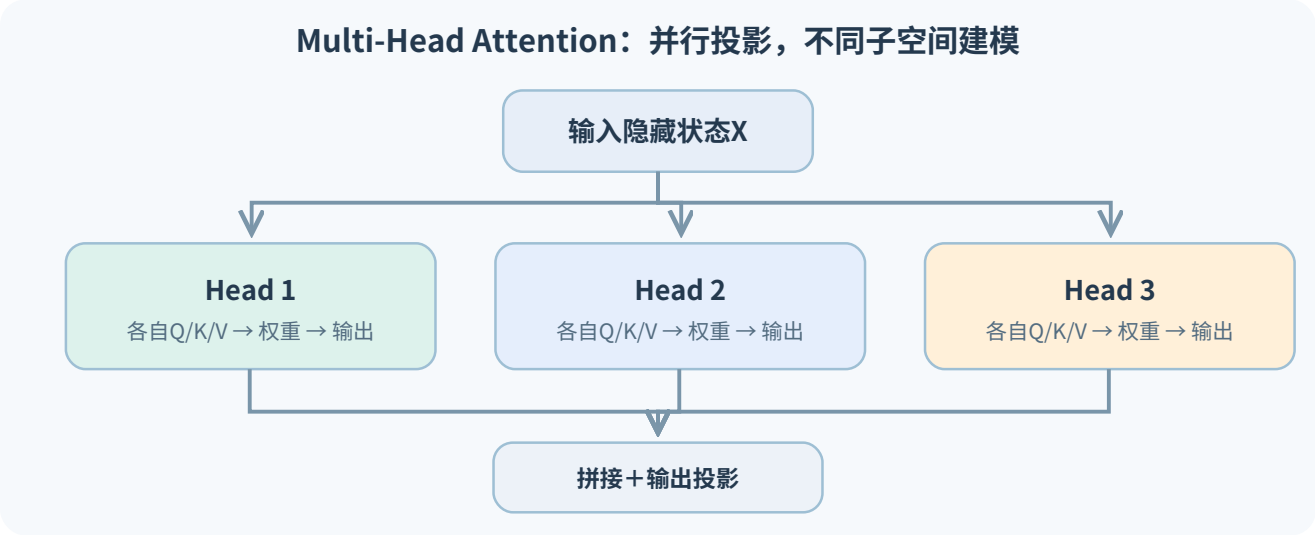


图6 · 多头提供不同表示空间，但不能给每个头预设固定的人类语义职责。

7.1 多头计算之后，还发生什么？

注意力并不是整个Transformer层。典型的Transformer块还会包含：

1. **输出投影**：把多个头的信息组合回隐藏表示空间。
2. **残差连接（Residual Connection）**：让网络把新计算结果与之前的表示相加，帮助信息传递和训练。
3. **归一化（Layer Normalization或RMSNorm等）**：调整数值尺度，帮助训练与计算稳定；不同模型实现不同。
4. **前馈网络（FFN/MLP）**：对每个位置的表示进行非线性变换，进一步形成复杂特征。

有的模型采用前归一化（Pre-Norm），有的架构不同，不能把某张教学流程图误当成所有当代大模型的统一实现。为零基础学习，你只需理解：**注意力主要负责跨位置的信息混合，前馈网络等模块进一步变换每个位置的表示；多层反复叠加，才形成复杂能力。**

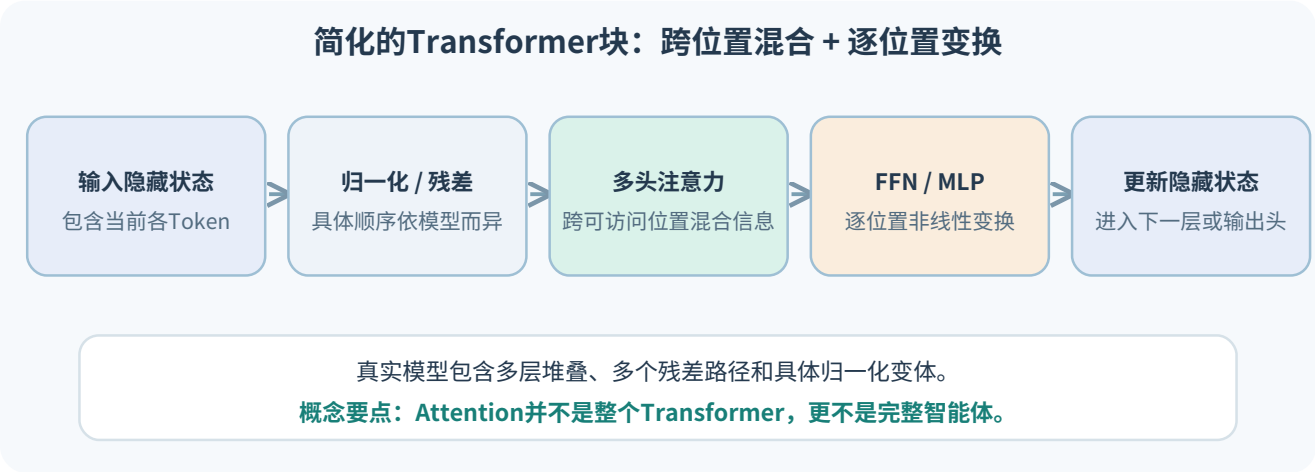


图7 · 教学顺序图仅展示组成部件，不固定真实网络的前后归一化实现。

7.2 为什么很多层比只有一层更有用?

第一次信息交换可能帮助“取消”建立与“方案”的关系，随后更高层可以将这段关系与“研究计划”及“当前问题”整合，再逐步形成下一Token所需的表示。这是便于教学的概念性推演，不是声称某一个真实模型精确按“第1层识别否定、第2层识别时间、第3层制定计划”的固定次序工作。真实网络层常存在复杂的分布式表示。

8. 位置如何影响注意力？为什么同样的事实换个地方会有区别?

只用词语集合，不能表达“先提出，再否定”与“先否定，再提出”的区别。所以模型需要使用某种位置信息。原始Transformer介绍过在输入表示中加入位置编码。[1] 之后提出了多种改进，包括RoPE：通过与位置有关的旋转方式，把位置差异引入查询与键的关系计算。[2]

8.1 位置不是简单的文件行号

模型实际处理的是Token位置以及模型内部由训练学到的位置效应。比如“第21天”这几个字本身也是语义信息，但“它出现在输入的前面还是后面”属于另一层位置关系。二者共同影响模型如何编码事实顺序。

8.2 长上下文的典型问题：Lost in the Middle

Liu等人的《Lost in the Middle》（TACL 2024）在特定的多文档问答与键值检索设置中发现：**相关证据位置改变，模型性能可能明显变化**；不少被测模型对放在开头或结尾的信息利用较好，中间的信息更容易出现性能下降。[3]

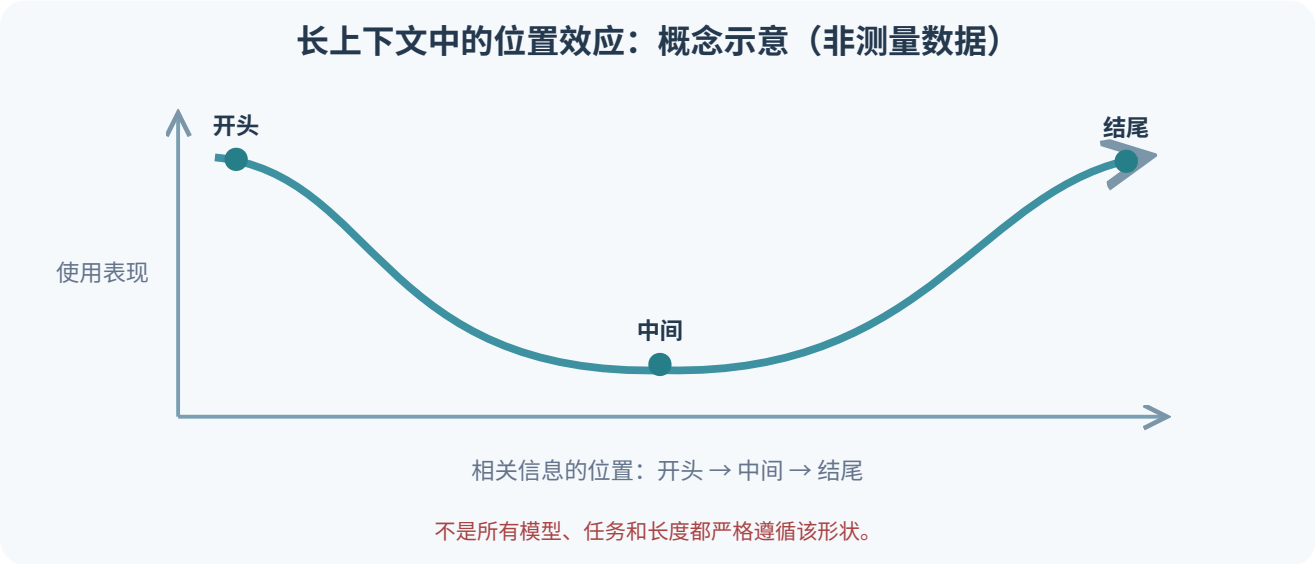


图8 · 文献报告过某些任务条件下的“中间较弱”现象，图形不代表原论文数据。

严格边界：图8是一张**定性概念示意图，不是论文真实实验数据，也不是“所有模型必然U形”的规律**。该研究报告的是特定模型、任务和实验条件下的现象。2024年的RULER还从多针检索、多跳追踪、聚合等角度说明：单个“针在大海捞针”成功，不足以证明全面的长上下文利用能力。[4]

8.3 为什么这对记忆系统特别重要?

当记忆系统检索出10条材料放进上下文，不能只检查“10条是否都装进去了”。还要考虑它们的组织、来源、时间状态以及模型能否正确综合。**检索召回率是前半程指标，最终正确决策是后半程指标。** 这也解释了为什么后续课程会单独讲检索排序、时间更新与系统评测。

9. 预填充、逐Token生成和KV Cache到底发生了什么?

把一次语言模型调用拆成两段，会更容易理解内部计算。

第一段：Prefill（预填充）。应用准备好提示词，模型在一次或分块的处理过程中计算输入Token的各层表示以及注意力所需的键和值。对于标准因果架构，前面位置不能偷看未来位置。

第二段：Decode（解码生成）。模型得到当前位置的输出概率分布，按采样或其他解码策略产生下一个Token。随后把新Token作为已生成前缀的一部分，继续产生下一个，直到结束条件触发。

9.1 KV Cache为什么能加速?

如果每生成一个Token都从头重新计算此前几千Token的所有键和值，会产生大量重复运算。KV Cache保存可复用的历史键和值等中间状态，使后续位置在注意力计算时使用这些缓存，而不必每次完全重做先前位置的投影与计算。

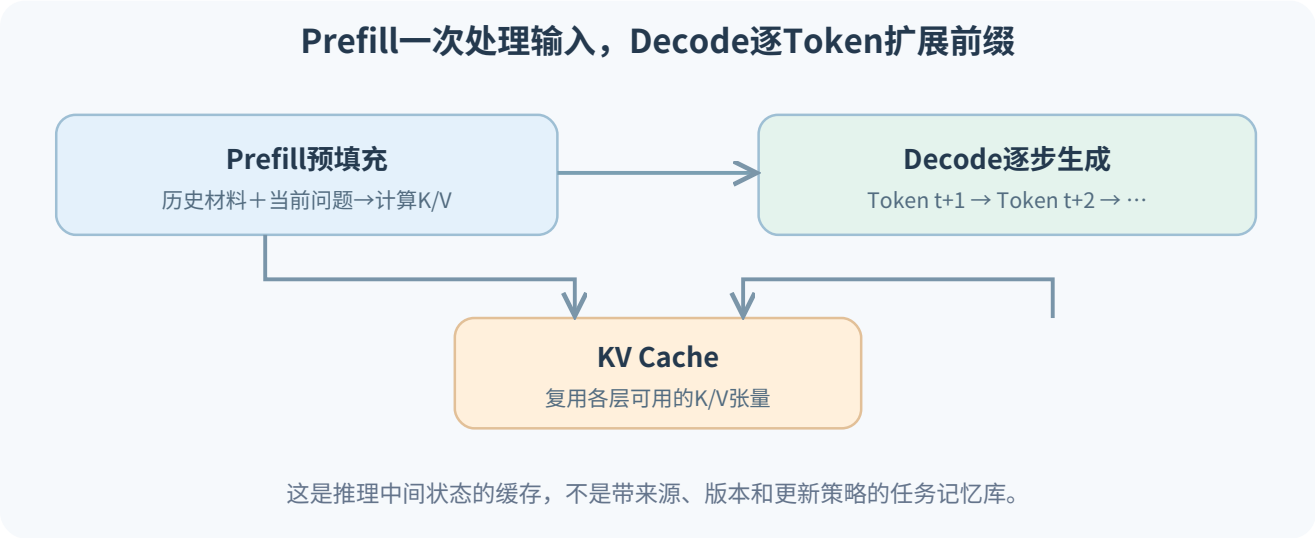


图9 · 缓存中间计算能减少重复运算，不会自动管理长期事实。

9.2 这个缓存与Agent Memory的差别

维度	KV Cache	外部智能体记忆
主要目的	复用模型推理中间状态，提高计算效率	使跨轮、跨任务事实与经验能够被维护和使用
典型内容	分层的键/值张量	历史事实、任务决策、经验、证据来源等
生命周期	依推理服务的缓存与会话策略	依应用级存储和治理策略，可长期保存
是否自带时间纠错	否	可以专门设计更新、冲突和失效机制
是否等于知识库检索	否	可以依赖检索器从存储中选择条目

KV Cache可以由特定推理系统跨请求复用，某些架构还研究跨片段递归状态；因此不能说它“绝对不跨请求”。但从功能职责看，**缓存中间张量并不会自动替代可维护的任务记忆**。Transformer-XL是研究跨片段依赖的一类早期代表，并不等于常见的外部语义记忆数据库。[5]

9.3 FlashAttention会让模型“记得更好”吗?

FlashAttention是一类**注意力计算的系统实现优化**，核心思路包含按块计算与减少GPU不同存储层级之间的读写，在经典设置中仍可实现精确注意力结果。[6] 它的目标首先是计算效率与内存访问效率。**优化计算实现≠自动解决过时信息、事实冲突和长期记忆检索的问题。** 将“能处理更长文本”和“能正确使用更长文本”混为一谈，是研究记忆系统时应避免的常见错误。

10. 注意力是一种“计算中的信息混合”，不是外部记忆检索

这句话是整门记忆课程的关键分界。我们把两件表面上都叫“找相关信息”的事情严格分开。

外部记忆检索：应用级系统收到当前任务，查询外部数据库、文件或索引，从大量**当前模型上下文之外**的候选记录中取回一些内容。可能使用BM25关键词匹配、向量相似度、结构化过滤、时间范围过滤或图遍历。其输出一般是可读文本、结构化数据或其他可以交给模型的证据。

模型内部注意力：输入已经组成一个Token序列，模型在Transformer内部按照训练学到的Q/K/V投影，在**当前可见序列表示**之间进行数值匹配、混合和变换。输出是新的隐藏向量，不是自动从硬盘取出来的旧聊天。

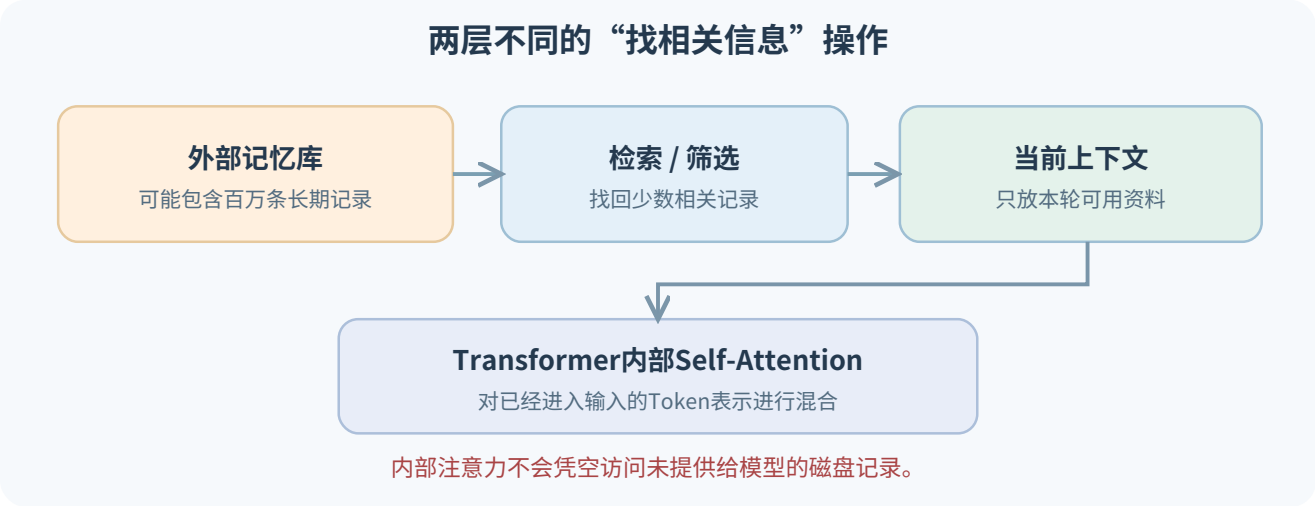


图10 · 检索决定输入范围，注意力参与输入范围内的计算。

10.1 两个“检索”最容易混淆的差别

外部检索器可以选择**哪些资料进入模型本轮计算**；而注意力处理的是**已经进入模型计算、符合掩码可访问规则的信息表示**。如果关键事实根本没有进入模型输入，普通自注意力无法凭空在外部数据库中把它读取出来。反过来，如果正确事实已经被检索回来，注意力与后续推理也不保证模型一定给出正确结论。

10.2 控制器必须承担哪些责任?

设第21天的正式决定为“当前不继续A路线”，第30天用户问“现在还做A吗？”一个可靠的长期Agent应当在模型调用前完成：

1. 确定当前问题与历史项目的对应关系，决定需要查找什么。
2. 从外部记忆中找到第7天旧提议、第18天反对证据、第21天正式决定。
3. 识别每条资料的时间、来源、效力和相互关系。
4. 选取必要的、可追溯的当前有效状态及支持证据，放入本轮上下文。
5. 让模型在该上下文中形成解释与建议，并按任务要求检查是否仍需外部核验。
6. 必要时保存新的确认决定，而不是把每一句模型建议都当成真实已执行决策。

划分责任很重要：第2步漏掉第21天决定，是检索器/索引/写入可能的问题；第4步放入大量重复旧稿遮蔽最新结论，是上下文选择问题；第5步误把“待评估”读成“已批准”，才是本轮模型解释与推理问题。没有区分，后面科研就无法做出可证伪的机制假设。

11. 让一个科研案例“从头到尾动一次”

我们固定一个例子，分别观察系统外部与模型内部发生什么。这是**教学构造的过程轨迹**，不代表某个真实产品暴露了完整内部状态。

第1阶段：长期记录在外部保存

第7天，系统保存：`A：候选假设；状态=待验证；提出时间=第7天`。

第18天，系统保存：`E：发现旧记忆可能诱发错误；状态=反例证据；来源=研究记录`。

第21天，系统保存：`D：不继续简单增加记忆数量；状态=当前正式决定；取代=A的直接执行提议`。

它们在物理存储上可能是三条JSON记录、数据库记录或文档段落。此处不预设“必须用向量数据库”。

第2阶段：第30天出现新问题

用户问：“我们下一步怎样推进？”Agent控制器需要先知道这是对**已有项目状态**的延续，而不是一个完全新项目。假设检索器找到了A、E、D三条记录，并将D标成当前有效，A标成旧提议。

第3阶段：组装上下文

为了让模型明确资料属性，可以把选出的记录整理成简短、带来源的输入：

> 项目当前目标：研究智能体记忆系统的失效机制。
> 历史提议A（第7天，待验证）：简单增加记忆数量。
> 反例E（第18天）：旧记忆可能使后续建议出错。
> 当前决定D（第21天，有效）：暂不继续简单扩大记忆数量，应先分析记忆更新与冲突。
> 用户问题（第30天）：我们下一步怎样推进？

这份上下文是**给模型看的材料**，不是数据库本身。需要注意，上面的“当前决定”和“有效”必须来自可信的业务状态或显式核验，不能由一段不受信任的外部网页随意自称。

第4阶段：Transformer对输入Token做处理

输入被Tokenizer切分，转成向量并带上位置关系。在各层带因果掩码的自注意力中，位于后面的用户问题和生成位置可以参考前面的任务目标、历史记录、反例和当前决定。在某些头或层的数值计算中，不同位置会产生不同的注意力权重，随后通过多层变换形成输出所需的隐藏状态。

必须保留证据边界：这段文字描述的是经典模型计算机制，不是声称我们能由一条输出直接观察到“模型内部第12层第7个头选中了D”。真实内部归因需要专门实验或可解释性方法。

第5阶段：模型生成建议，控制器负责验收与后续行动

假设模型回答：“下一步先分析旧记忆更新不及时和新旧结论冲突，不应直接扩大记忆数量。”这表明模型**在这一任务的最终输出上**与当前决定相符。它不证明所有相关记忆都被正确理解，也不证明模型已自动修改数据库。

如果下一步需要修改项目计划，应由控制器核实权限、执行对应工具，并根据**真实工具返回和授权决定**更新任务状态及记忆。**提出行动≠执行行动≠行动成功**；这是第04讲的系统职责原则，在注意力机制层同样适用。

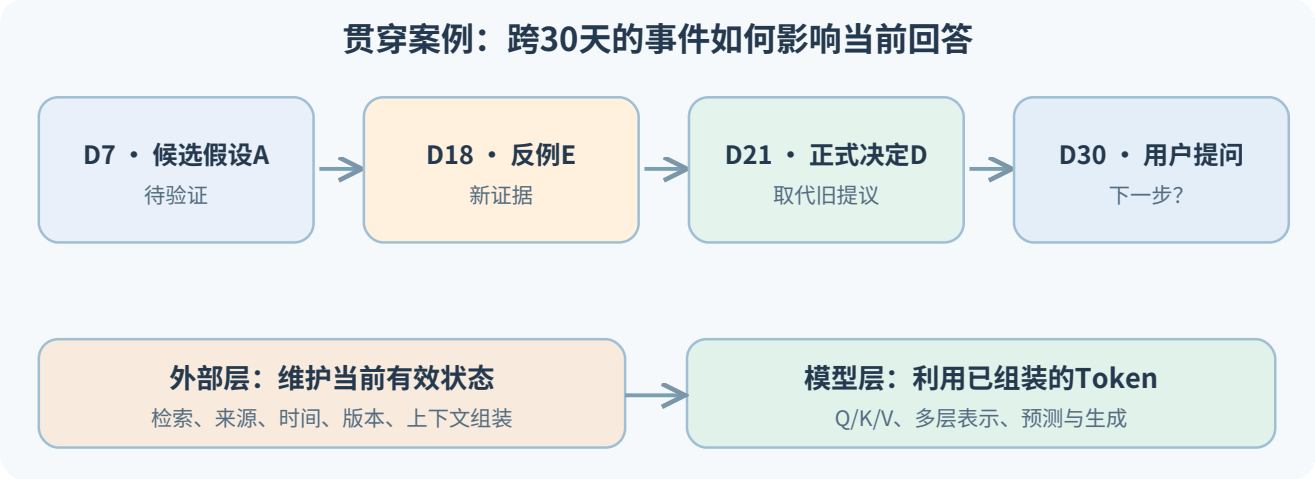


图11 · 同一项目有两层串联机制，不能把它们混成“模型记住了”。

12. 如果出现错误，应从哪一层开始排查?

我们用“旧方案误当成现行方案”做一个分层诊断。重点不是马上猜测“模型记性差”，而是排查信息在哪一层偏离了目标。

观察到的现象	优先检查	能支持怎样的结论
数据库里没有第21天的新决定	写入流程、持久化状态	可能属于记忆写入/保留缺陷
数据库有，但检索结果没找到	召回策略、索引、过滤条件	可能属于外部检索缺陷
检索到了，却没被放进模型输入	上下文选择、长度预算、截断	可能属于上下文组装缺陷
新旧记录均在，但没标清时间状态	状态/来源表示	可能属于记忆表示和治理缺陷
明确输入了当前决定，仍回答旧结论	模型解释、推理和指令遵循	可观察到上下文利用失败，但具体内部原因待分析
回答正确，执行了错误工具动作	控制器、工具参数、权限、验收	属于行动系统问题，不应全部归咎于记忆

12.1 为什么不能直接把注意力权重当成解释?

假设某个注意力头给第21天决定的权重较高。是否就证明模型会使用它来制定正确计划？不能。因为输出还经过其他头、其他层、前馈网络、残差连接与解码过程；单个权重不能独立决定最终结论。另一方面，一个头权重很小，也不能证明相关信息没有通过别的路径产生作用。

Jain与Wallace的论文《Attention is not Explanation》（NAACL 2019）曾用实验质疑“把注意力权重直接解释为模型决策证据”的做法。[7] 其后也有研究讨论何种条件下注意力可以具备解释价值。对本课而言，重要的实践原则是：**注意力图可以辅助分析，但仅凭热力图不能直接下内部因果结论。**

12.2 同样“答案错了”，可能不是注意力机制错了

正确事实可能被错误索引覆盖，可能被工具返回的旧缓存混入，可能在提示词中缺少来源，或者模型在最后一步把建议转成了未经授权的行动。研究时应先固定输入证据与操作环境，再判断错误到底发生在记忆的哪个阶段。否则即使换一种记忆结构后准确率变化，也难以确认变化的真正原因。



图12 · 不要把数据库、检索、模型推理和工具执行的问题统称为“注意力差”。

13. 常见误区：逐条对照消除混淆

误区1：“模型看到了所有Token，就一定可以准确利用其中任何一句。” 错。输入可见性只是前提，位置、干扰、推理复杂度、时间关系等都可能影响表现。长上下文研究已经展示过这种差别。[3][4]

误区2：“注意力就是找出最重要的一句话。” 不准确。经典注意力对各个可访问位置分配数值权重并混合Value，通常不是只选一个位置；数值权重也不是可信度或决策重要性的直接标签。

误区3：“Q是用户问题，K是向量数据库索引，V是原文。” 错。这只是便于理解的资料卡比喻。Q/K/V通常由层内隐藏向量经过学习到的矩阵投影得到。

误区4：“Softmax的权重加起来不等于1。” 在普通softmax对一组有限有效分数归一化时，精确权重和为1。教学中显示为0.999或1.001通常只是小数四舍五入。

误区5：“因果掩码会禁止模型使用历史信息。” 错。它禁止某个位置访问未来位置，当前位置仍可以使用此前符合条件的位置表示。

误区6：“模型内部注意力会自动搜索我硬盘上的旧聊天。” 错。普通注意力仅处理当前给它的表示；访问外部记录需要应用层的检索或工具流程。

误区7：“KV Cache就是Agent长期记忆。” 错。KV Cache主要保存推理中间状态；长期任务记忆还需要事实管理、检索和更新语义。

误区8：“多头注意力的一头专管否定，另一头专管时间，这是固定结构。” 错。不同头可能学到差异化行为，但不能按直观意图规定每个头的真实功能。

误区9：“Attention权重高，就说明模型真的相信这条证据。” 错。计算关联权重与证据真实性、来源权威、因果解释是不同的问题。[7]

误区10：“给模型更大上下文窗口，就不必建设记忆系统。” 错。更长输入有助于容纳历史，但不能自动解决跨任务持久性、信息压缩、时间更新、来源管理和主动检索策略。

误区11：“FlashAttention让模型具有更可靠的事实记忆。” 错。FlashAttention主要优化注意力的计算实现与内存读写，不直接构成事实的长期维护机制。[6]

误区12：“模型回答正确，说明数据库、检索器、注意力和行动控制全部没问题。” 错。一次正确输出只验证了该次输出；其他失败可能未被该样例触发，输出也不自动证明后续行动成功。

14. 理论思考题：先自己推导，不要急着看答案

第1题（概念）：为什么不能直接比较Token ID的大小来判断语义相似性？Embedding和隐藏状态分别起什么作用？

第2题（流程）：用户在第30天问“是否继续A项目”，系统数据库中有第21天的否定决定。请写出该决定最终能影响回答至少需要经过哪些环节。

第3题（QKV）：解释为何经典自注意力同一位置可以拥有三个不同的Q、K、V向量。它们来自几套可训练投影矩阵？

第4题（算术）：令 $q=[1,1]$，$k_1=[1,0]$，$k_2=[0,1]$，$k_3=[1,1]$，求未经缩放的点积分数，并判断哪个最大。

第5题（算术）：仍用第4题，$d_k=2$。请给出三个缩放分数，说明softmax后第3条的权重是否一定大于另两条，以及是否可以直接把它理解成“正确事实概率”。

第6题（加权求和）：假设三个权重分别为 $0.2、0.3、0.5$，Value分别为 $[1,0]、[0,1]、[1,1]$，注意力加权输出向量是多少？

第7题（因果掩码）：一个长度为5的序列使用标准因果掩码时，第2个位置最多能访问哪些位置？第5个位置呢？为什么第1个位置不应访问第5个？

第8题（架构）：为什么仅把一个Transformer注意力权重热力图贴在报告中，不能严谨地证明“某条历史记忆导致了错误行动”？

第9题（系统诊断）：数据库中保存了最新决定，检索也正确返回，但组装上下文时被截断。应先修改哪一层？单纯加大模型参数规模是否是可归因的修复？

第10题（跨层思考）：同时给模型提供旧提议和最新决定，与只提供最新决定相比，哪个一定更好？请从信息完整性、上下文干扰、可追溯性和任务需要四方面解释。

第11题（区别）：分别用一句话解释：外部记忆检索、Transformer自注意力、KV Cache、模型参数知识。再指出哪个最适合维护“用户昨天明确撤销的决定”。

第12题（迁移）：如果某模型在很长的“针在大海捞针”任务中做对了，但多跳历史推理做错了，是否能说它“没有长上下文能力”？应该怎样更加准确地描述？

15. 思考题参考答案：重点是推导逻辑

第1题答案：Token ID主要是词表中的离散编号。编号大小由词表与分配规则决定，数字差值通常不具备稳定的语义距离含义。Embedding把ID映射成可计算的向量；经过位置处理、注意力及前馈网络之后，隐藏状态变成对当前上下文更敏感的表征。不能把初始Embedding与最终上下文表示混为一谈。

第2题答案：首先要确保决定已正确写入持久存储；其次当前任务应触发或允许相关检索；检索器需召回第21天决定；上下文组装要把它连同必要时间与效力信息交给模型；模型再理解最新有效状态，形成回答；如回答将触发实际行动，控制器仍须按权限与工具反馈检查。每个步骤都可能失败，不能直接把最终输出错误归因于Attention。

第3题答案：典型做法是把每个位置的隐藏向量分别乘以 `W_Q`、`W_K`、`W_V` 三套不同的可训练矩阵，得到三组不同的投影。Q用于形成匹配请求，K用于参与匹配，V用于被加权汇总。同一个Token的语义角色可能因为投影空间不同而产生不同的数值表示。多头时每头还可能有各自对应的投影参数。

第4题答案：$q \cdot k_1 = 1\times1+1\times0=1$，$q \cdot k_2 = 1\times0+1\times1=1$，$q \cdot k_3 = 1\times1+1\times1=2$，因此第3条得分最高。这里使用的是人为二维数值，只能说明数学计算过程，不说明真实事实可信度。

第5题答案：缩放后为 $1/\sqrt{2}\approx0.707$、$1/\sqrt{2}\approx0.707$、$2/\sqrt{2}\approx1.414$。因为softmax对单调增大的分数赋予更高权重，第3条权重最大（约0.503），前两条各约0.248。权重是**数值匹配与归一化**结果，不是逻辑意义上的“正确事实概率”，更不是外部权限判断。

第6题答案：$0.2\times[1,0]+0.3\times[0,1]+0.5\times[1,1]=[0.2+0+0.5,\ 0+0.3+0.5]=[0.7,0.8]$。注意得到的是新向量，不是从三个Value里只挑出权重最高的一个。

第7题答案：第2个位置可访问位置1、2；第5个位置可访问1至5。第1个位置不能访问第5个，因为那相当于使用其后的未来Token信息，会破坏标准自回归“根据先前内容预测下一Token”的因果约束。已知输入可以被并行处理，但因果可见性仍需遵守这个规则。

第8题答案：模型的最终输出受多层、多头、残差、前馈网络、解码以及Agent控制器/工具共同影响。一个头的权重高或低，并不直接等于最终输出的因果贡献。应检查真实输入、对照与干预效果、输出及工具行为，必要时结合可解释性分析；单张热力图只能提供有限线索。[7]

第9题答案：应首先检查和调整**上下文选择、排序、压缩或截断策略**，让关键新决定真的进入模型本轮输入；必要时回到外部检索与预算设计。扩大模型参数规模未必修复组装环节遗漏，而且同时改变了底座模型，难以独立归因于被修复的机制。

第10题答案：不存在对所有任务都占优的答案。提供新旧两条记录利于解释决策变化、支持追溯，但会增加输入成本与冲突解释负担；只提供最新决定可降低干扰，但可能缺少“为什么否决”的证据。更合理的设计是基于任务需求提供**当前状态＋必要历史证据**，并标注时间和效力；不能把“越多越好”当作普遍规律。

第11题答案：外部记忆检索从模型当前上下文之外找相关历史记录；自注意力在本轮内部Token表示之间加权混合信息；KV Cache保存生成计算的中间键值等张量；模型参数知识来自训练形成的权重结构。维护“昨天撤销的决定”通常更适合有时间、来源和更新规则的应用级长期记忆系统；如果只依赖KV Cache或参数知识，不具备天然的撤销与版本治理语义。

第12题答案：不能一概说“完全没有长上下文能力”。更精确的描述是：它在所测的长文本单点定位任务中表现良好，但在多跳综合、时间状态或其他任务上出现了不足。应按任务类型、文本长度、位置、干扰程度、基线与测量方式报告。RULER等研究采用多任务评估，正是为了避免单指标代表整个能力。[4]

16. 把第05、06讲连接成一张完整的知识地图

第05讲讨论“能装多少”，第06讲讨论“装进去怎样被处理”。两者必须配在一起理解：

1. **输入容量关**：相关历史是否能够被放进当前上下文？由上下文窗口、控制器的选择/压缩等决定。
2. **内部表征关**：Token怎样变成向量、如何表示位置与关系？由Embedding、位置机制和Transformer等共同参与。
3. **信息混合关**：当前位置怎样结合历史位置？经典注意力用Q/K算权重、对V做加权汇总。
4. **推理与生成关**：多层计算怎样产出下一个Token？既要考虑上下文，也要考虑训练形成的模式与生成策略。
5. **行动与反馈关**：模型输出怎样成为工具行为？由控制器、权限、环境与真实反馈决定。
6. **跨时间记忆关**：必要事实如何从本轮之外重新进入未来本轮？要靠外部存储、检索、维护与治理。

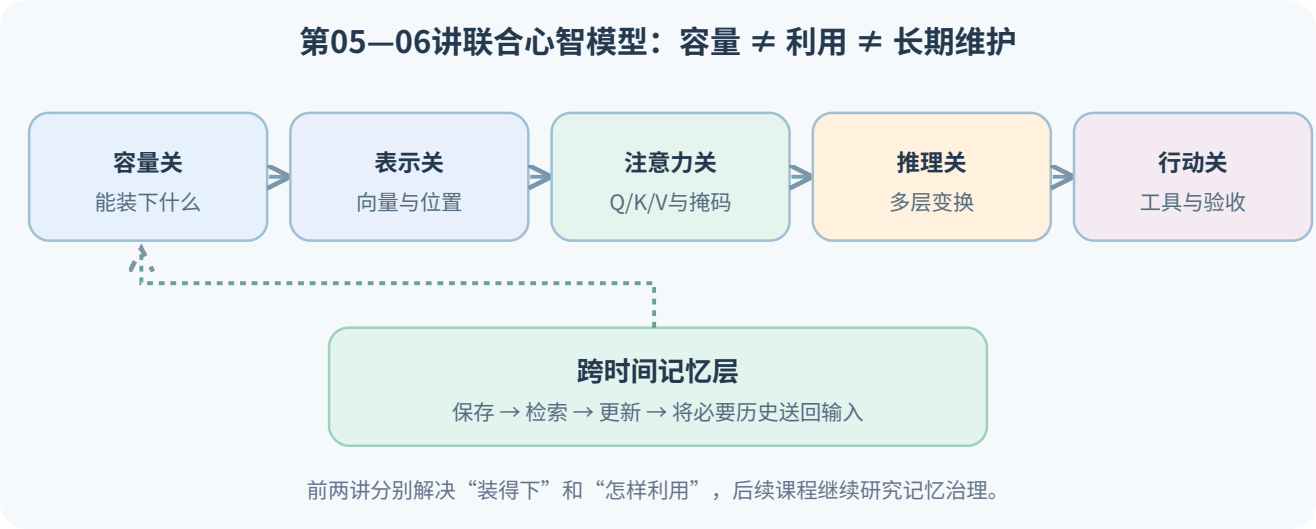


图13 · 把临时上下文计算与长期记忆管理放到同一张系统图中。

> **本讲最值得长期记住的一句话：外部记忆决定“哪些过去被带到眼前”；模型内部注意力参与“眼前这些Token如何交换信息”；控制器和环境决定“生成建议是否真正成为有效行动”。三个层次关联，却不能相互替代。**

17. 学习建议：不背词，尝试在脑中重放一次计算

请尝试不用笔记复述下面过程：

“第30天我问长期科研助手一个问题 → 控制器检索第21天的最新决定 → 把带来源的决定放进模型输入 → Token变成向量与位置表示 → 每层形成Q、K、V → 用Q/K算匹配分数、缩放、softmax → 用权重汇总V → 多层网络进一步变换 → 根据当前生成位置产生下一个Token → 控制器决定是否继续调用工具或更新任务状态。”

如果你能清楚说明每一步**是谁负责、输入是什么、输出是什么、下一步依赖什么**，你已经具备进入下一讲Embedding、语义向量与检索基础的必要心智模型。反过来，如果只能记住“Attention is all you need”或“QKV三个字母”，还不能算理解了运行机制。

18. 核对过的原始研究文献与延伸阅读

以下文献按其**适用的知识点**排列，并非要求零基础阶段立即通读。先读本文，理解机制后再看原文中的架构图与方法部分。带有“研究现象”的地方，应以各论文实际测试条件为边界。

1. **Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS.** Transformer、自注意力、Scaled Dot-Product Attention与Multi-Head Attention的经典出处。原文：https://arxiv.org/abs/1706.03762（会议PDF：https://papers.neurips.cc/paper/7181-attention-is-all-you-need.pdf）

2. **Su, J., et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding.** RoPE位置表示的原始研究。预印本：https://arxiv.org/abs/2104.09864
3. **Liu, N. F., et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. TACL 12:157–173.** 相关信息在长上下文不同位置时的表现差异。论文：https://aclanthology.org/2024.tacl-1.9/ （DOI:10.1162/tacl_a_00638）
4. **Hsieh, C.-P., et al. (2024). RULER: What's the Real Context Size of Your Long-Context Language Models?** 多样化长上下文能力评估。预印本：https://arxiv.org/abs/2404.06654
5. **Dai, Z., et al. (2019). Transformer-XL: Attentive Language Models beyond a Fixed-Length Context. ACL.** 跨片段依赖与内部状态复用思路。论文：https://aclanthology.org/P19-1285/
6. **Dao, T., et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS.** 经典注意力计算的IO效率优化。预印本：https://arxiv.org/abs/2205.14135
7. **Jain, S., & Wallace, B. C. (2019). Attention is not Explanation. NAACL.** 对“注意力权重直接等于解释”的实证讨论。论文：https://aclanthology.org/N19-1357/；补充学术争论可参见Wiegreffe & Pinter (2019) Attention is not not Explanation：https://aclanthology.org/D19-1002/

课程下一讲：第07讲《Embedding与语义向量：文字如何成为可比较的数字？》。本讲解释的是模型内部注意力的基础；下一讲会把向量的几何意义、相似度和外部语义检索联系起来，从而为后续向量记忆系统作铺垫。